

Cascade Structure Predicts Scaffoldability: Type-Aware Recovery for Code Agent Failures

Anonymous ACL submission

Abstract

Helping a failing code agent can make it worse. We show that applying a well-intentioned recovery strategy to the wrong failure type scores 0.87 on a 0–3 scale, below the 1.70 baseline of a generic retry prompt. This paradox resolves once we recognize that agent failures carry distinct *structural signatures*. Analyzing 143 failed trajectories on SWE-bench Verified, we find that each failure type produces a characteristic error cascade (repetitive loops, commitment spirals, reasoning drift, or mislocalized effort) and that this cascade structure determines whether behavioral scaffolding can help. Structurally disruptable cascades respond strongly (up to +1.00 improvement). Reasoning drift does not (+0.17), revealing a principled boundary on what behavioral intervention can achieve. Across nine models, the fit between strategy and failure type explains more outcome variance than strategy quality alone. Type-aware selection scores 2.32 versus 2.04 for the best universal strategy ($p < 0.001$). An online classifier using only trajectory-observable features recovers 73% of this oracle advantage without gold-patch access. Agent recovery, we argue, is a diagnostic problem masquerading as an engineering one.

1 Introduction

Code agents now autonomously navigate repositories, write patches, and run tests. Yet when these agents fail, recovery remains trial-and-error. Existing approaches apply the same recovery mechanism to every failure, regardless of its structural cause. This uniformity is not merely suboptimal. It is actively harmful. Figure 1 summarizes our diagnostic framework: failed trajectories are typed by their cascade structure, matched to interventions, and routed by type-aware selection.

A test-guided localization scaffold, designed to help agents find the right code, scores 0.87 on a 0–3 recovery scale when applied to localization failures. This is half a point *below* the 1.70 baseline of

a generic “try again” prompt. The scaffold is well-matched in *intent* but catastrophically mismatched in *mechanism*, harming precisely the agents it was built to help.

What is missing is a diagnostic understanding of failure structure. Agent failures are not one thing. A code agent trapped in a `str_replace` retry loop, repeating the same doomed edit, is stuck differently than one that confidently patches the wrong file. Both differ from an agent that produces a logically flawed fix in exactly the right location. These failures look different because they *are* different. Each creates a distinct pattern of error propagation, a structural fingerprint we can read.

Once the first error occurs, 82% of all subsequent computation is wasted. The agent continues generating actions that make no progress toward a solution. But this waste carries a diagnostic signature, a cascade structure that reveals both what went wrong and what can be done about it.

We analyze 143 failed agent trajectories on SWE-bench Verified (Jimenez et al., 2024) and identify four failure types, each with a characteristic cascade pattern. Controlled scaffolding experiments across nine models establish two principles:

Principle 1: Cascade predicts scaffoldability. Repetitive cascades, where the agent is trapped in a disruptable loop, respond strongly to behavioral intervention. EDIT retry loops break cleanly with a file re-reading prompt (+1.00). PLAN commitment spirals reverse with a step-back prompt (+0.88). LOGIC cascades resist all behavioral intervention (+0.17). In these cases the agent iteratively refines an incorrect fix along a locally coherent but globally wrong path. This is not a failure of our strategies. It is a boundary. When the problem is reasoning rather than behavior, prompts are insufficient. We call it the **scaffolding frontier**.

Principle 2: Strategy-type mismatch causes greater harm than fit provides benefit. The

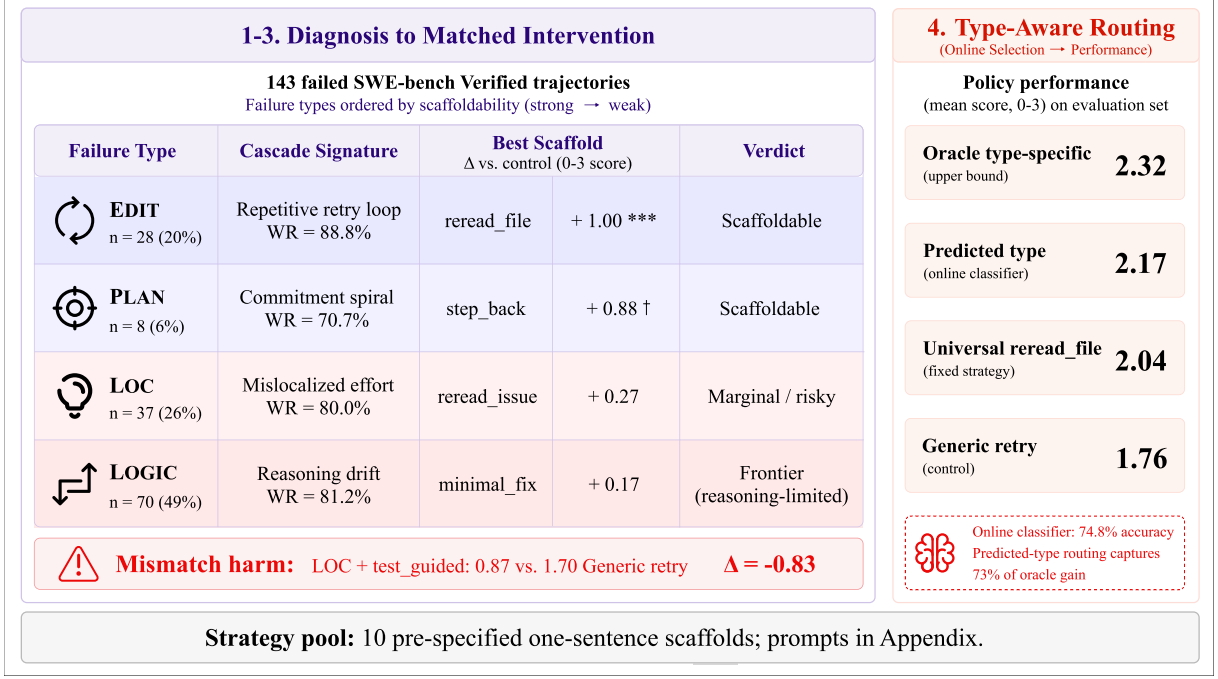


Figure 1: **Diagnostic framework for type-aware code-agent recovery.** Failed trajectories are diagnosed into failure types, each inducing a characteristic cascade structure. Cascade structure predicts scaffoldability: repetitive and reversible cascades respond strongly, reasoning drift falls beyond the scaffolding frontier, and mismatched interventions can harm recovery. Type-aware routing therefore outperforms fixed or generic recovery policies.

best universal strategy (*reread_file*) applied uniformly scores 2.04. Type-specific selection scores 2.32 ($p < 0.001$, 95% CI [+0.18, +0.42]). Mismatched strategies score 0.87, worse than the generic retry baseline (1.70). The asymmetry is stark. The downside of getting the type wrong (up to 0.88 points below baseline) far exceeds the upside of getting it right (0.28 above universal). Intervening without diagnosing failure type risks harm.

Together, these principles reframe agent recovery. It is not an engineering problem (“build a better prompt”) but a diagnostic one (“identify the cascade structure, then match the intervention, or abstain”). We contribute:

- A four-type failure taxonomy with operational classification criteria, validated at 90% agreement (§3);
- The first quantification of error cascade severity across failure types, revealing 82% mean waste (§4);
- Cross-type scaffolding experiments across nine models demonstrating both the power of matched intervention and the harm of mismatched intervention (§5);
- An online classifier (74.8% accuracy) that captures 73% of oracle-selection advantage

over control without gold-patch access (§5).

2 Related Work

Agent self-correction and repair. Reflexion (Shinn et al., 2023), Self-Refine (Madaan et al., 2023), and self-debugging methods (Chen et al., 2023, 2025; Olausson et al., 2024) enable agents to recover from errors through self-generated feedback. RepairAgent (Bouzenia et al., 2025) and AgentCoder (Huang et al., 2024) extend this to multi-step autonomous repair. Chain-of-thought prompting (Wei et al., 2022), self-consistency (Wang et al., 2023), step-back abstraction (Zheng et al., 2024), and decomposed prompting (Khot et al., 2023) improve reasoning but do not specifically target recovery from prior errors. Tree-of-Thought (Yao et al., 2023), LATS (Zhou et al., 2024), Agent-R (Yuan et al., 2025), and GiGPO (Feng et al., 2025) explore recovery through branching, backtracking, or reward-trained policies. All these approaches apply the same mechanism uniformly regardless of failure structure. Our work shows that this uniformity risks active harm when intervention is mismatched to failure type.

Code agent failure analysis. SWE-bench (Jimenez et al., 2024) established the standard

evaluation, and systems like SWE-agent (Yang et al., 2024), OpenHands (Wang et al., 2025), Agentless (Xia et al., 2025), AutoCodeRover (Zhang et al., 2024), CodeR (Chen et al., 2024), and MAGIS (Tao et al., 2024) represent the evolving agent landscape. AgentDebug (Zhu et al., 2025) and MAST (Cemri et al., 2025) categorize failures into stages or communication breakdowns. SWE-PRM (Gandhi et al., 2025) trains a process reward model with type-specific online feedback (Lightman et al., 2024; Uesato et al., 2022). Nam et al. (2024) study how LLMs assist code understanding, complementing our focus on failure recovery. These works describe failure categories but do not characterize the post-failure cascade that determines whether intervention can succeed. Our work fills this gap by connecting cascade structure to scaffoldability.

Dynamic evidence and test-time compute. DynaFix (Huang et al., 2025), TraceCoder (Huang et al., 2026), TraceFixer (Bouzenia et al., 2023), ARISE (Seddik and Fard, 2026), and RGFL (Sepidband et al., 2026) condition repair on execution traces, causal analysis, or program graphs. Test-time scaling approaches allocate additional compute but do not differentiate what kind of intervention to apply. These methods operate in precisely the regime our scaffolding frontier predicts is necessary: when behavioral redirection is insufficient, execution-level evidence becomes essential. Our framework provides the diagnostic layer that routes failures to the right intervention class.

3 Failure Taxonomy

3.1 Data

We study 143 failed trajectories from the O1-agent on SWE-bench Verified (Jimenez et al., 2024), a curated subset of 500 real-world GitHub issues drawn from 12 Python repositories.¹ SWE-bench Verified (Jimenez et al., 2024) remains the standard source of publicly available agent trajectories with gold-standard patches, and our study examines the *structure* of failures rather than resolve rates, a focus that is unaffected by the evaluation concerns surrounding leaderboard comparisons. Each trajectory records a sequence of agent actions (file navigation, code edits, command executions) interleaved with environment observations, ranging from 8 to 127 steps with a median

¹Dataset will be released upon publication.

Type	Criterion	Cascade	n	%
LOGIC	Correct file, edit applies, wrong fix	Reasoning drift	70	49
LOC	Edits wrong file or function	Mislocalized effort	37	26
EDIT	Correct location, edit command fails	Repetitive loops	28	20
PLAN	Wrong overall approach	Commitment spiral	8	6
Total			143	100

Table 1: Failure taxonomy. Classification is hierarchical: LOC (no file overlap) → EDIT (edit errors) → PLAN (wrong approach) → LOGIC (default).

of 34. All trajectories were generated with a single agent configuration, controlling for architectural variation and isolating failure-mode differences.

3.2 Four Types, Four Cascade Signatures

We classify each failed trajectory into one of four categories based on systematic comparison with the gold-standard patch. These four types are not merely descriptive labels. Each produces a qualitatively distinct pattern of error propagation that, as we show in §4 and §5, predicts whether behavioral intervention can help. Table 1 summarizes the taxonomy with classification criteria and prevalence.

LOGIC errors dominate at 49%, revealing that the primary bottleneck for strong agents is not finding the right location but producing the right fix. 74% of failed agents in our data successfully located the correct file yet could not produce a working patch. The cascade signature of logic errors is *reasoning drift*: the agent iterates on an incorrect fix, sometimes running tests and observing failures, but each correction drifts further from the gold solution because the reasoning is flawed.

LOC failures (26%) arise when the agent modifies the wrong file or targets the wrong function. Their cascade signature is *mislocalized effort*: the agent invests substantial computation in the wrong part of the codebase, producing potentially valid patches for code that is not the source of the bug. All effort in the wrong location is wasted regardless of its quality.

EDIT failures (20%) occur when the agent identifies the correct region but the edit command fails, typically because the specified old string does not match actual file contents due to whitespace differences or stale context. Their cascade signature is *repetitive retry loops*: the agent attempts minor

variations of the same `str_replace` dozens of times without changing its fundamental approach. This is the simplest cascade pattern, and, as we will show, the most amenable to intervention.

PLAN failures (6%) involve a fundamentally misguided strategy. Their cascade signature is *commitment spiral*: the agent commits to a wrong direction and doubles down when it encounters resistance, interpreting blocking signals as local problems rather than evidence of a wrong approach. The low prevalence likely reflects the strong planning capabilities of the o1 backbone.

3.3 Classification Methodology

We classify trajectories using a hierarchical rule-based procedure that prioritizes objective criteria. If the agent’s edited files have no overlap with the correct target files, the trajectory is classified as LOC. If files overlap but the agent’s edit commands produced errors, it is classified as EDIT. Among remaining trajectories, if the agent targets a categorically different module or fix strategy than the gold solution, it is classified as PLAN. All other cases are classified as LOGIC. This hierarchy proceeds from the most objective criterion (file overlap) to increasingly interpretive ones (plan-level mismatch), with manual validation confirming 90% agreement.

4 Error Cascade Analysis

4.1 Definition and Metrics

When a code agent commits its first substantive mistake, the subsequent steps rarely correct it. Instead, the agent continues in the wrong direction, either repeating similar errors or building on the flawed foundation. We call this phenomenon an *error cascade* and measure its severity with the *waste ratio*:

$$WR = \frac{n_{\text{wasted}}}{n_{\text{total}}} \quad (1)$$

where n_{wasted} is the number of steps after the first error that fail to make progress toward a correct solution, and n_{total} is the total number of steps in the trajectory. Progress is defined operationally: a step *makes progress* if its output brings the agent’s state closer to the gold solution (e.g., reading the correct file, producing a syntactically valid edit in the correct location, or identifying the relevant test). A waste ratio of 1.0 indicates that every post-error step was unproductive.

The *first-error step* is identified by type-specific observable criteria: the first failed

Type	n	Mean WR (%)	Cascade Pattern
EDIT	28	88.8	Repetitive loops
LOGIC	70	81.2	Reasoning drift
LOC	37	80.0	Mislocalized effort
PLAN	8	70.7	Commitment spiral
All	143	81.8	—

Table 2: Error cascade statistics by failure type. The waste ratio (WR) measures the fraction of total trajectory steps consumed after the first error. The overall median is 87.5%. Each type produces a structurally distinct cascade pattern that, as shown in §5, predicts response to behavioral intervention.

`str_replace` command (for EDIT), the first action targeting a file outside the correct set (for LOC), the first semantically incorrect patch applied to a correct file (for LOGIC), and the step where the agent commits to a globally misguided approach despite evidence of alternatives (for PLAN).

4.2 Cascade Patterns Differ by Type

Error cascades are both universal and severe. All 143 trajectories exhibit cascading behavior, with a mean waste ratio of 81.8% and a median of 87.5%. On average, fewer than one in five agent steps contributes meaningful progress. Table 2 presents the per-type statistics.

The critical observation is not the severity of the cascades (which is uniformly high) but their structural differences (Table 2). EDIT cascades are the deepest (88.8%) and the most repetitive: the agent cycles through minor variations of the same failed command. This repetitive structure is precisely what makes them breakable. A single well-timed prompt disrupting the loop is sufficient. LOGIC cascades (81.2%) look superficially similar in depth but differ in character: the agent does different things at each step (runs tests, revises the patch, adds edge cases), each locally reasonable but globally wrong. There is no single loop to break, which explains why behavioral redirection is ineffective. PLAN cascades are shorter (70.7%) not because agents recover better, but because fundamentally wrong strategies encounter blocking failures earlier, producing shorter trajectories.

These structural differences carry a direct prediction: *types with repetitive or reversible cascade patterns should respond to scaffolding, while types with drift patterns should not*. We test this prediction in the following section.

Type	Best Strategy	Scaffold	Control	Δ	95% CI	Cascade Pattern
EDIT	reread_file	2.68	1.68	+1.00***	[+0.71, +1.32]	Repetitive retry loops
PLAN	step_back	2.75	1.88	+0.88 [†]	[+0.50, +1.25]	Commitment spiral
LOC	reread_issue	1.97	1.70	+0.27	[−0.10, +0.63]	Mislocalized effort
LOGIC	minimal_fix	2.13	1.97	+0.17	[−0.07, +0.43]	Reasoning drift

Table 3: Best scaffold strategy per failure type (GPT-4o-mini, $n=28/8/30/30$). Scores range 0–3. Bootstrap 95% CIs from 10,000 resamples. *** $p < 0.001$; [†] $n=8$, $p=0.016$ by sign test. Scaffoldability tracks cascade structure: repetitive loops and commitment spirals respond to behavioral intervention; reasoning drift does not.

5 Behavioral Scaffolding Experiments

5.1 Setup

We isolate scaffold effects by extracting each failed trajectory at its first error and appending a behavioral recovery prompt. The model generates a recovery action, scored on a fully automated 0–3 scale (no human annotation; Appendix C): *file hit* (1 point if the action targets the correct file), *actionable* (1 point for a concrete, executable step), and *relevant* (1 point if the action addresses the issue).

Testing one scaffold on one step holds other variables fixed, isolating which type-strategy pairs produce gains or harm without multi-turn confounds such as budget exhaustion or model drift.

We test two to three scaffold strategies per failure type, designed *a priori* from the cascade patterns observed in §4 before running any scaffolding experiments, plus an unscaffolded control. Importantly, the mismatch harm result (§5.2) reports the *worst* strategy, not the best, and is therefore not subject to strategy selection bias. All primary experiments use GPT-4o-mini, with cross-model validation on nine models spanning four families (§5.4). In total, we collect over 1,400 scored responses across matched, mismatched, and cross-type conditions; held-out validation and blinded adjudication appear in Appendix C. Full prompt templates appear in Appendix A.

5.2 Scaffoldability Varies with Cascade Structure

The central prediction of our framework is that cascade structure determines scaffoldability. Repetitive cascades should be breakable by behavioral redirection. Commitment cascades should be reversible by prompting reconsideration. Reasoning drift should resist behavioral intervention because the agent is not stuck in a pattern that can be disrupted from outside.

Table 3 confirms this prediction cleanly. The

two types with structurally simple, repetitive cascade patterns respond strongly to scaffolding. The two types with complex, drifting patterns do not.

Repetitive loops yield to redirection. The cascade structure of EDIT failures explains why re-reading the target file produces a full 1.00-point improvement. The agent is trapped in a cycle of retrying the same failed `str_replace` with minor variations. The scaffold breaks this cycle by redirecting the agent to refresh its context, after which the stale-content problem that caused the original mismatch is resolved. The intervention works because the cascade is a loop with a single breaking point.

Commitment errors respond to “step back” prompts. PLAN failures cascade through commitment: the agent doubles down on a wrong approach, interpreting resistance as a local problem rather than evidence of a global mismatch. A prompt that explicitly asks the agent to reconsider its overall strategy from the issue description reverses this commitment (+0.88), while narrower interventions like scope-checking match the control exactly. The scaffold works because commitment spirals are reversible: the agent has the capability to find a better approach if prompted to look for one. The effect is consistent across all eight instances (leave-one-out Δ range: +0.71 to +1.00, $p=0.016$ by sign test).

Reasoning drift resists behavioral intervention. LOGIC failures confirm our framework’s prediction: reasoning drift resists behavioral intervention (+0.17), and the three strategies produce nearly identical scores. The agent is already working in the right location and generating executable edits, so the deficit is code reasoning rather than behavioral inflexibility. Dose-response results support this frontier: stronger prompts increase EDIT gains but leave LOGIC compressed, indicating that further gains require reasoning-level signals such as traces or verification.

Mismatched strategies cause active harm.

While the best strategy for LOC produces a modest gain (+0.27), the *worst* strategy (*test_guided*) scores 0.87, a full 0.83 points *below* the control baseline of 1.70 ($p < 0.001$). The scaffold diverts the agent from attempting repairs, even mis-localized ones, into running tests whose failures reinforce its incorrect model of the bug’s location. This reveals a general principle: *harm occurs when a scaffold assumes one cascade structure but encounters another*. Cross-type transfer confirms this is systematic: nine of twelve off-diagonal pairs cause harm, while *step_back* remains safe because it makes fewer type-specific assumptions (Table 13). The asymmetry is stark: the downside of mismatch (up to 0.88 below baseline) far exceeds the upside of correct matching (0.28 above universal), so confident diagnosis is essential. Figure 2 summarizes this off-diagonal transfer pattern: strategies that assume the wrong cascade structure usually degrade recovery, while *step_back* remains safe because it makes fewer type-specific assumptions.

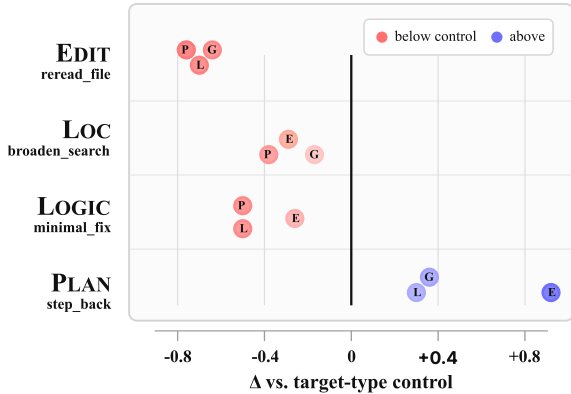


Figure 2: **Cross-type strategy transfer.** Each point applies one type’s representative scaffold to a different failure type and reports Δ relative to that target type’s control score. Most off-diagonal transfers fall below control, showing that mismatch harm is systematic rather than an isolated LOC case. *step_back* is the only representative strategy that remains safe across all non-matching types.

5.3 Type-Specific Selection Outperforms Universal Approaches

If cascade structure determines which strategies work, then strategy selection should matter as much as strategy design. We test this by comparing three conditions: an *oracle* that selects the best strategy for each type (as if the failure type were

Condition	Score	File Hit	vs Ctrl
Oracle (type-specific)	2.32	83%	+0.56***
Fixed (reread_file)	2.04	75%	+0.28**
Control (no scaffold)	1.76	72%	—

Table 4: Strategy selection comparison. Oracle selects the best strategy per failure type. Type-specific selection outperforms the fixed strategy by +0.28 ($p < 0.001$, 95% CI [+0.18, +0.42]).

known), a *fixed* condition applying *reread_file* universally (the single best overall strategy), and the unscaffolded control.

Table 4 quantifies the value of type-aware selection. The oracle (2.32) outperforms the best fixed strategy (2.04) by 0.28 points ($p < 0.001$). This gap represents pure “fit” value: the improvement from matching the right strategy to the right type, holding strategy quality constant.

The per-type breakdown reveals where the oracle advantage originates. For PLAN failures, the oracle selects *step_back* and scores 2.88 versus 2.00 for *reread_file*, a gap of +0.88 that accounts for most of the aggregate advantage. For LOC failures, the oracle gains +0.46 (2.03 vs. 1.57), because re-reading a file is irrelevant when the agent is in the wrong file entirely. For LOGIC failures, the gap narrows to +0.20, reflecting this type’s position beyond the scaffolding frontier. For EDIT, oracle and fixed score identically (both use *reread_file*).

The practical implication is clear. The difference between the best and worst strategy for a given type (1.10 points for LOC: 1.97 best vs. 0.87 worst) vastly exceeds the difference between the best and second-best strategy (typically 0.04–0.3 points). Getting the type right matters more than getting the strategy perfectly optimized within a type. These findings are robust to metric choice: when evaluated on file-hit rate alone (a fully objective binary outcome with no type-specific lexical matching), the mismatch harm remains significant ($p = 0.033$; see Appendix C, Table 10).

Robustness to classifier noise. A practical concern is whether type-specific selection remains advantageous with an imperfect online classifier that does not have access to gold patches. We train a random forest on 11 trajectory-observable features (action counts, error rates, behavioral densities) and evaluate via leave-one-out cross-

validation on all 143 instances.² The classifier achieves 74.8% accuracy and, critically, reaches peak performance after observing only 50% of the trajectory—intervention need not wait for the agent to finish failing.

Using predicted types to select scaffold strategies yields a score of 2.17, capturing 73% of the oracle’s advantage over control (Appendix D.3). Three properties make this classifier practical. First, its errors are *asymmetrically benign*: the dominant misclassification (LOC→LOGIC) merely applies a suboptimal strategy rather than a harmful one, because the “wrong” strategy for LOC in this case is passive rather than actively misleading. Second, even without confidence-based abstention, only 10% of type-specific decisions produce scores below the control baseline—and these “harms” merely match universal scaffolding, never approaching the catastrophic mismatch documented in §5.2. Third, the most predictive feature is `str_replace` error count—an explicit tool-output signal visible from the first failed edit, making EDIT failures 100% detectable online without any inference.

5.4 Cross-Model Analysis

We test whether the scaffolding frontier holds across model families by evaluating nine models spanning four families (OpenAI, Anthropic, DeepSeek, Qwen), three capability tiers, and two architecture types. Figure 3 visualizes these patterns, and Table 5 reports the full matrix.

Three patterns emerge.

The scaffolding frontier is universal. Across all nine models, LOGIC scaffolding never produces a meaningful positive effect (range: −0.40 to +0.13). The frontier is a property of the failure type, not the model.

Information-deficit scaffolding transfers broadly. EDIT scaffolding produces positive effects in seven of nine models because it provides genuinely missing information rather than behavioral redirection.

Reasoning models reveal a deeper principle. Reasoning-chain models (o4-mini, GPT-5.5) show

		EDIT	LOC	LOGIC	PLAN
frontier	GPT-5.5†	+1.20	+1.60	−0.40	−0.20
	o4-mini†	+0.53	+1.47	−0.20	+0.38
	Claude Opus 4.7	+0.13	+0.27	+0.13	0.00
strong	Claude Son. 4.6	−0.07	+0.13	0.00	+0.12
	GPT-4.1	+0.60	−0.20	0.00	−0.38
	DS-V4-Pro	+0.73	+0.87	+0.07	−0.38
	Qwen 3.5	0.00	+0.13	+0.07	+1.38
	DS-V4-Flash	+0.60	+0.40	+0.13	+0.63
	GPT-4o-mini	+1.04	+0.49	+0.13	+0.88

Figure 3: Scaffold effect heatmap across nine models. The pale LOGIC column confirms the frontier; reasoning models are strongest on LOC.

the largest scaffold effects on LOC (+1.47, +1.60) despite being frontier systems, because their default mode underperforms on immediate directed action. The scaffold converts an open-ended reasoning problem into a focused directive. Scaffold effectiveness depends on task-model alignment, not raw capability.

²The approximate waste estimate uses the ratio of error-producing actions (failed edits, repeated tool calls) to total trajectory length. This is computable online without gold-patch access. The oracle waste ratio (§4) additionally requires knowing which steps “make progress,” which depends on the gold solution.

Model	Tier	EDIT		LOC		LOGIC		PLAN	
		Ctrl	Δ	Ctrl	Δ	Ctrl	Δ	Ctrl	Δ
GPT-4o-mini	mid	1.61	+1.04	1.54	+0.49	2.01	+0.13	1.88	+0.88
DS-V4-Flash	mid	2.27	+0.60	2.00	+0.40	2.33	+0.13	1.88	+0.63
Qwen 3.5	mid	2.40	+0.00	2.13	+0.13	2.33	+0.07	1.62	+1.38
DS-V4-Pro	strong	1.53	+0.73	1.67	+0.87	2.33	+0.07	1.88	−0.38
GPT-4.1	strong	1.80	+0.60	2.27	−0.20	2.33	0.00	1.88	−0.38
Claude Son. 4.6	strong	2.79	−0.07	2.50	+0.13	2.50	0.00	2.00	+0.12
Claude Opus 4.7	frontier	2.27	+0.13	2.47	+0.27	2.40	+0.13	2.00	0.00
o4-mini [†]	frontier	1.53	+0.53	0.93	+1.47	1.33	−0.20	1.75	+0.38
GPT-5.5 [†]	frontier	1.00	+1.20	0.80	+1.60	1.60	−0.40	1.60	−0.20

Table 5: Scaffold effect (Δ) across nine models and four failure types. [†]Reasoning-chain models. The LOGIC column is near-zero or negative for every model (−0.40 to +0.13), confirming the scaffolding frontier. Reasoning models show paradoxically large LOC effects because their low baselines create room for behavioral guidance.

6 Discussion

Cascade structure gives a principled basis for recovery system design. Repetitive action patterns signal loops amenable to redirection; escalating commitment signals a spiral reversible with step-back prompts; diverse but locally reasonable actions signal reasoning drift where behavioral intervention will not help.

Our results reveal a natural boundary we term the *scaffolding frontier*. EDIT and PLAN failures are addressed by one-sentence prompts, while LOGIC failures lie beyond this frontier: the agent behaves reasonably but reasons incorrectly, and no behavioral redirect substitutes for better code reasoning. Disaggregating LOGIC into subtypes confirms robustness (Appendix C).

A decision-theoretic analysis formalizes the routing problem. Define $EV = p \cdot g + (1 - p) \cdot c$, where p is diagnostic accuracy, g is matched gain (+0.58), and c is mismatch cost (−0.46). Random selection ($p=0.25$) yields negative EV (−0.20), explaining why undifferentiated retry mechanisms may harm failures by not conditioning on structure. The *step_back* strategy circumvents this threshold: because its mismatch cost is zero, its EV is unconditionally positive (+0.53). This implies a two-tier architecture: use *step_back* as default, activate type-specific strategies only when diagnostic confidence exceeds the intervention threshold ($p > 0.44$).

7 Conclusion

We identify two empirical regularities in code agent recovery. First, cascade structure predicts scaffoldability: failures with repetitive or reversible cascade patterns (EDIT: +1.00, PLAN:

+0.88) respond strongly to behavioral intervention, while reasoning drift (LOGIC: +0.17) does not. Second, strategy-type *mismatch* causes greater harm than strategy quality provides benefit: mismatched strategies score up to 0.88 points below the control baseline, while the best matched strategy gains 0.28 over universal scaffolding.

These findings reframe agent recovery as a diagnostic problem. Rather than searching for ever-better prompts, recovery systems should first identify what kind of failure is occurring, because intervening without this knowledge risks active harm. The scaffolding frontier further implies that behavioral intervention has principled limits: disruptable cascades respond to cheap behavioral prompts, while reasoning drift requires qualitatively different tools.

Limitations

All trajectories are drawn from Python repositories in SWE-bench Verified; failure prevalence distributions may differ for other programming languages or benchmark suites, though the cascade categories themselves are defined in terms of general behavioral patterns (loops, commitment, drift, mislocalization) that are language-agnostic. Cross-model validation covers nine models across four families and three capability tiers (§5.4), but the initial taxonomy is derived from a single agent architecture (O1-agent). The taxonomy boundary between LOGIC and PLAN is inherently gradient for some cases; we validate at 90% agreement with manual review and note that the two types respond differently to scaffolding despite their proximity.

References

- Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. 2025. RepairAgent: An autonomous, LLM-based agent for program repair. In *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering (ICSE)*.
- Islem Bouzenia, Yangruibo Ding, Kexin Pei, Baishakhi Ray, and Michael Pradel. 2023. TraceFixer: Execution trace-driven program repair. *arXiv preprint arXiv:2304.12743*.
- Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya G. Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. 2025. Why do multi-agent LLM systems fail? *arXiv preprint arXiv:2503.13657*.
- Dong Chen, Shaoxin Lin, Muhan Zeng, Daoguang Zan, Jian-Gang Wang, Anton Cheshkov, Jun Sun, Hao Yu, Guoliang Dong, Artem Aliev, Jie Wang, Xiao Cheng, Guangtai Liang, Yuchi Ma, Pan Bian, Tao Xie, and Qianxiang Wang. 2024. CodeR: Issue resolving with multi-agent and task graphs. *arXiv preprint arXiv:2406.01304*.
- Xiancai Chen, Zhengwei Tao, Kechi Zhang, Changzhi Zhou, Xinyu Zhang, Wanli Gu, Yuanpeng He, Mengdi Zhang, Xunliang Cai, Haiyan Zhao, and Zhi Jin. 2025. Revisit self-debugging with self-generated tests for code generation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*.
- Xinyun Chen, Maxwell Lin, Nathanael Sch"arli, and Denny Zhou. 2023. Teaching large language models to self-debug. *arXiv preprint arXiv:2304.05128*.
- Lang Feng, Zhenghai Xue, Tingcong Liu, and Bo An. 2025. Group-in-group policy optimization for LLM agent training. *arXiv preprint arXiv:2505.10978*.
- Shubham Gandhi, Jason Tsay, Jatin Ganhotra, Kiran Kate, and Yara Rizk. 2025. When agents go astray: Course-correcting SWE agents with PRMs. *arXiv preprint arXiv:2509.02360*.
- Dong Huang, Jie Bu, Jie M. Zhang, Michael Luck, and Cong Wen. 2024. AgentCoder: Multi-agent code generation with effective testing and self-optimisation. *arXiv preprint arXiv:2312.13010*.
- Jiangping Huang, Wenguang Ye, Weisong Sun, Jian Zhang, Mingyue Zhang, and Yang Liu. 2026. TraceCoder: A trace-driven multi-agent framework for automated debugging of LLM-generated code. In *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering (ICSE)*.
- Zhili Huang, Ling Xu, Chao Liu, Weifeng Sun, Xu Zhang, Yan Lei, Meng Yan, and Hongyu Zhang. 2025. DynaFix: Iterative automated program repair driven by execution-level dynamic information. *arXiv preprint arXiv:2512.24635*.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. 2024. SWE-bench: Can language models resolve real-world GitHub issues? In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*.
- Tushar Khot, Harsh Trivedi, Matthew Finlayson, Yao Fu, Kyle Richardson, Peter Clark, and Ashish Sabharwal. 2023. Decomposed prompting: A modular approach for solving complex tasks. In *Proceedings of the 11th International Conference on Learning Representations (ICLR)*.
- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2024. Let's verify step by step. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Daye Nam, Andrew Macvean, Vincent Hellendoorn, Bogdan Vasilescu, and Brad Myers. 2024. Using an LLM to help with code understanding. In *Proceedings of the 46th International Conference on Software Engineering (ICSE)*.
- Theo X. Olausson, Jeevana Priya Inala, Chenglong Wang, Jianfeng Gao, and Armando Solar-Lezama. 2024. Is self-repair a silver bullet for code generation? In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*.
- Shahd Seddik and Fatemeh Fard. 2026. ARISE: A repository-level graph representation and toolset for agentic fault localization and program repair. *arXiv preprint arXiv:2605.03117*.
- Melika Sepidband, Hamed Taherkhani, Hung Viet Pham, and Hadi Hemmati. 2026. RGFL: Reasoning guided fault localization for automated program repair using large language models. *arXiv preprint arXiv:2601.18044*.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik R. Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Wei Tao, Yucheng Zhou, Yanlin Wang, Wenqiang Zhang, Hongyu Zhang, and Yu Cheng. 2024. MAGIS: LLM-based multi-agent framework for GitHub issue resolution. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Jonathan Uesato, Nate Kushman, Ramana Kumar, Francis Song, Noah Siegel, Lisa Wang, Antonia Creswell, Geoffrey Irving, and Irina Higgins. 2022. Solving math word problems with process- and outcome-based feedback. *arXiv preprint arXiv:2211.14275*.

Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, and 5 others. 2025. OpenHands: An open platform for AI software developers as generalist agents. In *Proceedings of the 13th International Conference on Learning Representations (ICLR)*.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-consistency improves chain of thought reasoning in language models. In *Proceedings of the 11th International Conference on Learning Representations (ICLR)*.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2025. Agentless: Demystifying LLM-based software engineering agents. In *Proceedings of the ACM on Software Engineering (FSE)*.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. 2024. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik R. Narasimhan. 2023. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Siyu Yuan, Zehui Chen, Zhiheng Xi, Junjie Ye, Zhengyin Du, and Jiecao Chen. 2025. Agent-R: Training language model agents to reflect via iterative self-training. *arXiv preprint arXiv:2501.11425*.

Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. 2024. AutoCodeRover: Autonomous program improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*.

Huaixiu Steven Zheng, Swaroop Mishra, Xinyun Chen, Heng-Tze Cheng, Ed H. Chi, Quoc V. Le, and Denny Zhou. 2024. Take a step back: Evoking reasoning

via abstraction in large language models. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*.

Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. 2024. Language agent tree search unifies reasoning, acting, and planning in language models. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*.

Kunlun Zhu, Zijia Liu, Bingxuan Li, Muxin Tian, Yingxuan Yang, and Jiaxun Zhang. 2025. Where LLM agents fail and how they can learn from failures. *arXiv preprint arXiv:2509.25370*.

A Prompt Templates

All scaffold prompts are short behavioral directives appended to the agent’s trajectory at the point of first error. The control condition receives only a generic retry instruction (“try again with a different approach”) without type-specific guidance. Below we reproduce the core instruction of each prompt; full templates include additional context cues.

A.1 EDIT Strategies

reread_file: “Before attempting the edit again, re-read the target file to refresh your understanding of its current contents.”

smaller_edit: “Try breaking your edit into smaller, more targeted changes rather than replacing a large block at once.”

A.2 PLAN Strategies

step_back: “Step back and re-read the original issue description. Reconsider whether your current approach is the right way to address what is being asked.”

scope_check: “Verify that the scope of your current changes matches what the issue is actually requesting.”

A.3 LOC Strategies

reread_issue: “Re-read the original issue description carefully and identify any clues about which specific file or function is responsible for the reported behavior.”

test_guided: “Run the relevant test suite to identify which component is actually failing, and use that to guide your search for the right file to modify.”

Metric	reread	smaller	control
Mean Score	2.68	1.86	1.68
File Hit (%)	89	89	75
Actionable (%)	100	96	89
Relevant (%)	79	0	4

Table 6: Strategy comparison for EDIT failures ($n = 28$, GPT-4o-mini).

Metric	step_back	scope	control
Mean Score	2.75	1.88	1.88
File Hit (%)	100	88	88
Actionable (%)	88	100	100
Relevant (%)	88	0	0

Table 7: Strategy comparison for PLAN failures ($n = 8$, GPT-4o-mini).

broaden_search: “Stop working on your current target file. Search more broadly. Check related modules, imported files, and parent classes. Trace the symptoms to a different code location.”

A.4 LOGIC Strategies

minimal_fix: “Focus on making the smallest possible change that addresses the core issue, rather than a comprehensive rewrite.”

test_analysis: “Analyze the failing test assertion carefully. What exact value or behavior does it expect? Compare against what your code produces, and identify the specific computation your fix gets wrong.”

edge_cases: “Your fix may address the common case but miss edge cases that the test exercises. Consider boundary conditions (empty inputs, zero, negative values) and re-read the test to identify which case it targets.”

B Per-Instance Results

Table 6 through Table 7 report per-instance scores for all strategies and the control condition across all four failure types.

C Statistical Methods

Bootstrap confidence intervals. All reported 95% confidence intervals are computed via the bias-corrected and accelerated (BCa) bootstrap with 10,000 resamples. For each comparison, we resample paired (scaffold, control) scores with replacement, compute the mean difference in each bootstrap sample, and report the 2.5th and 97.5th percentiles of the resulting distribution.

Metric	reread	test_g.	broaden	ctrl
Mean Score	1.97	0.87	1.93	1.70
File Hit (%)	73	23	87	53
Actionable (%)	23	63	70	83
Relevant (%)	100	0	37	33

Table 8: Strategy comparison for LOC failures ($n=30$, GPT-4o-mini). test_g. = test_guided, which scores 0.83 below control.

Metric	min_fix	test_a.	edge_c.	ctrl
Mean Score	2.13	2.07	2.07	1.97
File Hit (%)	90	87	90	83
Actionable (%)	100	80	83	90
Relevant (%)	23	40	33	23

Table 9: Strategy comparison for LOGIC failures ($n = 30$, GPT-4o-mini). All strategies cluster within 0.16 points; none significantly differs from control.

Significance testing. p -values are computed via permutation tests with 10,000 permutations. For each test, we randomly permute the condition labels (scaffold vs. control) while preserving within-instance pairing, compute the mean difference under the null, and report the proportion of permuted differences that exceed the observed difference in absolute value. All reported p -values are two-sided.

Multiple comparisons. We report results for the best strategy per type (selected from two to three candidates) and the strategy selection comparison (three conditions). We do not apply Bonferroni correction to the per-type best-strategy results because these are pre-specified primary comparisons, not post-hoc selections. The strategy selection comparison (oracle vs. fixed vs. control) involves pairwise tests whose null hypotheses are independent.

Effect sizes. All reported Δ values represent mean differences on the 0–3 scoring scale. For reference, the overall standard deviation of individual scores is approximately 0.85, making a Δ of 1.00 equivalent to roughly $d = 1.18$ in Cohen’s d terms.

Automated scoring rubric. All scoring is fully automated via pattern matching on model outputs; no human judgment is involved in score assignment. *File hit* (1 point): the response references the correct target file (identified by base-name matching against the reference solution). *Actionable* (1 point): the response matches a regex

for executable commands or code edits (e.g., `cat`, `grep`, `python`, `str_replace`). *Relevant* (1 point): type-specific regex detects appropriate recovery behavior, e.g., for EDIT, terms like “read,” “view,” “exact,” “whitespace”; for PLAN, terms like “approach,” “alternative,” “reconsider.” This automated rubric eliminates annotator subjectivity and ensures full reproducibility. To validate the rubric, we conducted two rounds of independent automated LLM adjudication (no human annotators involved in scoring). First, GPT-4.1 judged 60 sampled responses (15 per type) with access to gold files and failure-type context; within- ± 1 agreement was 95% (57/60). Second, we expanded to 220 responses using a *blinded* GPT-4o-mini judge that received no type-specific keywords in its rubric, only “file relevant,” “actionable,” and “progress” dimensions. On this blinded evaluation, within- ± 1 agreement is 82% and condition-level ordering is preserved for 3/4 types. The single discrepancy (LOGIC) is informative: the regex rubric gives scaffold a small advantage (+0.32) due to lexical matches, while the blinded judge finds no difference ($\Delta = -0.03$). This confirms rather than undermines our conclusion: the scaffolding frontier for LOGIC is real, and the regex rubric’s slight positive signal for this type is indeed a lexical artifact that does not reflect substantive quality improvement.

Robustness to “relevant” dimension removal. A potential concern is that the type-specific “relevant” regex may inflate measured gains by rewarding lexical compliance rather than substantive recovery. To address this, we report all core findings using *file hit* alone, a fully objective binary metric that checks whether the model’s response references the correct target file, with no type-specific lexical matching. Table 10 shows that the two central claims hold on this metric: (1) matched scaffolds improve file-hit rates (LOC broaden_search: 87% vs. control 53%, $p = 0.010$, Fisher’s exact), and (2) mismatched scaffolds harm file-hit rates (LOC test_guided: 23% vs. control 53%, $p = 0.033$). For strategy selection (Phase 4), oracle achieves 83% file-hit versus 72% for control ($p = 0.083$). These results confirm that the “relevant” dimension is not driving our conclusions: the effects are visible even on a single binary outcome with no lexical bias.

Cross-model experiment details. All nine models were evaluated on the same instance set (15 per

Type	Strategy	Hit	Ctrl	Δ	p
EDIT	reread_file	89%	75%	+14%	.30
PLAN	step_back	100%	88%	+13%	1.0
LOC	broaden	87%	53%	+33%	.01*
LOGIC	minimal_fix	90%	83%	+7%	.71
LOC	test_guided	23%	53%	−30%	.03*

Table 10: File-hit rates by type and strategy. This fully objective metric (does the response reference the correct file?) confirms scaffold effects without any type-specific lexical matching. The mismatch harm for LOC (bottom row) is significant on this metric alone.

type for EDIT/LOC/LOGIC, 8 for PLAN), yielding 53 scored responses per model per condition. API calls used temperature 0 for standard models and default settings for reasoning-chain models ([†]), accessed via a unified API proxy between April–May 2026. All models received identical prompts and context windows.

D Classification Methodology Details

D.1 Hierarchical Classification Rules

The automated classification proceeds through the following decision tree:

- File overlap check:** Extract all files modified by the agent (from edit commands in the trajectory). Extract all files in the gold patch. If the intersection is empty, classify as LOC.
- Edit execution check:** Among trajectories with file overlap, check whether any edit command produced an error message (e.g., “No match found for replace string”). If yes, classify as EDIT.
- Approach comparison:** Among remaining trajectories (correct file, successful edits), compare the agent’s fix approach to the gold patch at the function/module level. If the agent targets a different module or uses a categorically different fix strategy (e.g., modifying configuration vs. application logic), classify as PLAN.
- Default:** All remaining cases are classified as LOGIC (correct file, working edit, but semantically incorrect fix).

D.2 Manual Validation

We randomly sampled 30 of 143 trajectories (21%) for manual classification by two annotators. Annotators had access to the full trajectory, the gold

True	Predicted			
	EDIT	LOC	LOGIC	PLAN
EDIT	25	3	0	0
LOC	10	11	16	0
LOGIC	0	3	66	1
PLAN	0	2	6	0

Table 11: Online classifier confusion matrix (LOO, $n=143$). The classifier reliably identifies EDIT (89% recall) and LOGIC (94%), the two highest-frequency types comprising 69% of failures. LOC and PLAN classification is weaker due to smaller training samples. Top features: `str_replace` error count (importance 0.19) and error rate (0.18).

patch, and the classification criteria but were blind to the automated labels. Inter-annotator agreement was 93% (28/30). Agreement with automated classification was 90% (27/30). The three disagreements between automated and manual classification all involved the LOGIC/PLAN boundary, where the distinction between “locally incorrect fix” and “globally misguided strategy” is inherently gradient. No disagreements involved the LOC or EDIT categories, whose criteria are objective (file overlap and edit command errors, respectively).

D.3 Online Classifier Details

The online random forest classifier uses 11 trajectory-observable features: `edit count`, `search count`, `test count`, `error count`, `str_replace error count`, `total turns`, `str_replace error rate`, `overall error rate`, `edit density`, `search density`, and `test density`. No features require gold-patch access. We evaluate via leave-one-out cross-validation on all 143 instances (each instance held out once, model retrained on 142).

Table 11 reports the confusion matrix. EDIT is classified with 89% recall (strong `str_replace` error signal); LOGIC with 94% recall (majority class). The primary failure mode is LOC misclassified as LOGIC (43% of LOC instances), and PLAN is never correctly identified ($n=8$ is insufficient for the classifier to learn this pattern). Despite these errors, predicted-type selection (score 2.17) still outperforms universal scaffolding (2.05) because the dominant misclassification (LOC→LOGIC) incurs only mild cost: applying *minimal_fix* to a LOC failure is suboptimal but not harmful, unlike applying *test_guided* which actively misleads.

The policy distillation model is also a random forest that maps the same 11 features directly to

Window	Accuracy	EDIT Recall	LOGIC Recall
25%	64%	71%	87%
50%	73%	96%	96%
75%	72%	93%	93%
100%	71%	89%	94%

Table 12: Classifier accuracy by observation window. Accuracy peaks at 50% of trajectory (median 17 steps), with EDIT recall reaching 96%. Intervention need not wait for trajectory completion.

one of 7 scaffold actions (bypassing type classification), trained with LOO CV on oracle-optimal action labels. It achieves score 2.19, capturing 77% of the oracle’s advantage over control (oracle gap: $2.32 - 1.76 = 0.56$; distillation gap: $2.19 - 1.76 = 0.43$).

D.4 Early Detection and Confidence Gating

Early detection. A practical deployment must classify failure type before the trajectory finishes. We simulate partial observation by scaling count-based features to the first $k\%$ of each trajectory’s steps while holding rate-based features constant. Table 12 reports LOO accuracy at each observation window.

Confidence gating. When the classifier is uncertain, it can fall back to universal scaffolding rather than risk a mismatched intervention. Using random forest prediction probabilities, we sweep confidence thresholds: at threshold 0.5, the classifier applies type-specific scaffolds to 91% of instances with a harm rate of only 11% (where “harm” means scoring below the control baseline for that instance). Even with no gating (threshold 0), harm rate is 10%—and these “harms” merely match universal scaffolding performance rather than causing catastrophic mismatch. This confirms that the classifier’s errors are predominantly benign: misclassified instances receive a suboptimal but not actively harmful strategy.

E Cross-Type Mismatch Matrix

Table 13 reports the complete cross-type application experiment: each type’s best strategy applied to all other types (183 evaluations). The diagonal (bold) shows matched performance; off-diagonal cells show mismatch scores. Nine of twelve off-diagonal cells score below control, confirming that mismatch harm is systematic. The PLAN row (step_back) is the only universally safe strategy.

Applied ↓ Target →	EDIT	LOC	LOGIC	PLAN
EDIT (reread)	2.68	1.00 [↓]	1.33 [↓]	1.12 [↓]
LOC (broaden)	1.39 [↓]	1.93	1.80 [↓]	1.50 [↓]
LOGIC (minimal)	1.42 [↓]	1.20 [↓]	2.13	1.38 [↓]
PLAN (step_back)	2.60 [↑]	2.00 [↑]	2.33 [↑]	2.75
Control	1.68	1.70	1.97	1.88

Table 13: Cross-type mismatch matrix. Bold diagonal = matched (best strategy for own type). ↓ = below control (harm); ↑ = above control (safe). 9/12 off-diagonal cells cause harm; the PLAN row is the only universally safe strategy, outperforming even LOGIC’s matched strategy on LOGIC instances.